

# Quantifying Label Leakage in Code Vulnerability Datasets: A Follow-up Ablation Study

Turing  
RTSS Board  
turingrtss@gmail.com

September 2026

## Abstract

In a prior study, we identified label leakage in a public code vulnerability dataset: a TF-IDF classifier’s top predictive features were substrings of the words “vulnerable” and “safe” appearing in code comments. We hypothesized that this leakage substantially inflated the reported 84.7% accuracy. This follow-up ablation study tests that hypothesis by progressively removing leakage sources — comments, label-correlated words, and identifier names — and retraining. The fully cleaned model retains 81.1% accuracy, indicating that only 3.6 percentage points (4.3% relative) were attributable to label leakage. After cleaning, the model’s top features are genuine code patterns: `eval()`, string concatenation operators, and conditional structures. We revise our prior conclusion: while label leakage is present and should be controlled for, the model learns substantial real vulnerability patterns even in uncleaned data.

## 1 Introduction

In our prior work [1], we trained TF-IDF classifiers on the `Code_Vulnerability_Labeled_Dataset` [2] and achieved 84.7% binary classification accuracy. Feature analysis revealed that the model’s strongest predictive signals were character n-gram substrings of the words “vulnerable” and “safe,” which appeared in comments and variable names. We concluded that label leakage inflated the reported accuracy and suggested it would “drop significantly” after cleaning.

This follow-up study directly tests that prediction through a controlled ablation: we progressively remove potential leakage sources and measure the accuracy impact of each removal. The goal is to quantify how much of the original accuracy was genuine versus artifactual.

## 2 Methods

### 2.1 Dataset and Model

We use the same dataset (8,480 samples, 14 classes, binary split: 4,461 safe / 4,019 vulnerable) and model (TF-IDF character n-grams 3–6, logistic regression,  $C = 1.0$ ) as the original study, with identical train/test split (80/20, stratified, seed 42).

### 2.2 Ablation Conditions

Five conditions were tested, each applying cumulative cleaning:

1. **Baseline:** Original code, unmodified.
2. **Comments stripped:** All single-line comments (`#`, `//`), multi-line comments (`/* */`), docstrings (`""" """`), and lines consisting entirely of natural language text removed.

3. **Label words removed:** A curated list of 20 label-correlated words (“vulnerable,” “safe,” “injection,” “overflow,” “exploit,” “malicious,” “sanitize,” etc.) removed from the code regardless of context.
4. **Both:** Comments stripped and label words removed.
5. **Full clean:** Comments stripped, label words removed, and identifiers normalized (function names replaced with `func_N`, string literals replaced with `"STR"`).

### 2.3 Metrics

For each condition, we report classification accuracy, vulnerable-class F1 score, false positive rate, and the top 5 predictive features for each class. Feature type is categorized as *Code* (executable syntax) or *NL* (natural language artifact).

## 3 Results

Table 1: Ablation results: accuracy under progressive cleaning.

| Condition           | Accuracy | Drop (pp) | Vuln F1 | FP Rate |
|---------------------|----------|-----------|---------|---------|
| Baseline            | 0.847    | —         | 0.842   | 16.7%   |
| Comments stripped   | 0.826    | −2.1      | 0.823   | 20.1%   |
| Label words removed | 0.839    | −0.8      | 0.834   | 17.6%   |
| Both combined       | 0.826    | −2.1      | 0.822   | 19.8%   |
| Full clean          | 0.811    | −3.6      | 0.805   | 20.3%   |

### 3.1 Feature Shift

After full cleaning, the top predictive features changed substantially from the baseline:

Table 2: Top 5 vulnerability features: baseline vs. fully cleaned.

| Rank | Baseline              |        | Full Clean            |      |
|------|-----------------------|--------|-----------------------|------|
|      | Feature               | Type   | Feature               | Type |
| 1    | <code>' + '</code>    | Ambig. | <code>' + '</code>    | Code |
| 2    | <code>' vu'</code>    | NL     | <code>' eval('</code> | Code |
| 3    | <code>' vuln'</code>  | NL     | <code>' eval'</code>  | Code |
| 4    | <code>' vul'</code>   | NL     | <code>' eva'</code>   | Code |
| 5    | <code>' this '</code> | NL     | <code>' ev'</code>    | Code |

Table 3: Top 5 safety features: baseline vs. fully cleaned.

| Rank | Baseline             |      | Full Clean           |      |
|------|----------------------|------|----------------------|------|
|      | Feature              | Type | Feature              | Type |
| 1    | <code>' sa'</code>   | NL   | <code>' if'</code>   | Code |
| 2    | <code>' the'</code>  | NL   | <code>' if '</code>  | Code |
| 3    | <code>' the'</code>  | NL   | <code>' if '</code>  | Code |
| 4    | <code>' safe'</code> | NL   | <code>' els'</code>  | Code |
| 5    | <code>' saf'</code>  | NL   | <code>' else'</code> | Code |

After cleaning, all top-5 features in both classes correspond to executable code constructs. The vulnerability side is dominated by `eval()` and string concatenation. The safety side is dominated by conditional logic (`if/else`), suggesting that safe code contains more defensive branching.

## 4 Discussion

### 4.1 Revising the Original Conclusion

Our prior study suggested that label leakage would cause accuracy to “drop significantly” after cleaning. The measured drop of 3.6 percentage points (4.3% relative) does not support this prediction. The model retains 81.1% accuracy using only code-structural features, indicating that TF-IDF character n-grams capture genuine vulnerability patterns in this dataset.

We overestimated the leakage impact for two reasons:

1. We conflated feature *rank* with feature *contribution*. Label-word substrings were the highest-ranked features, but they did not dominate the overall classification boundary, which is defined by thousands of features.
2. We assumed that natural language features contributed no signal. In practice, comments describing vulnerabilities may co-occur with the vulnerabilities themselves, providing weakly valid (if non-generalizable) signal.

### 4.2 What the Model Actually Learns

The cleaned model’s features reveal three genuine vulnerability patterns:

- **eval() family:** The use of `eval()` or `exec()` is a well-known dangerous pattern.
- **String concatenation (+):** Building strings dynamically is a precursor to injection vulnerabilities (SQL, command, template).
- **Absence of conditionals:** Safe code contains more `if/else` branching, consistent with defensive input validation.

These patterns are real and would transfer to production code, supporting the use of TF-IDF classifiers as a lightweight first-pass vulnerability screen.

### 4.3 Decomposition of Accuracy Loss

The 3.6 pp total drop decomposes as:

- Comment removal:  $-2.1$  pp (largest single source)
- Label word removal:  $-0.8$  pp (smaller than expected)
- Identifier normalization:  $-1.5$  pp (additional, from condition 4 to 5)
- Combined effect is not purely additive (conditions 2+3 individually sum to 2.9 pp, but combined yield 2.1 pp), indicating overlap between comment-based and word-based leakage.

## 4.4 Limitations

1. Comment stripping is imperfect — some natural language may survive in string literals or logging statements.
2. The label word list (20 words) may be incomplete; other domain-specific terms could carry leakage.
3. Identifier normalization is regex-based, not AST-based. A proper parser would produce cleaner normalization.
4. The dataset contains multiple programming languages. Language-specific cleaning would be more thorough.

## 5 Conclusion

Label leakage in the tested vulnerability dataset accounts for 3.6 percentage points of the 84.7% baseline accuracy — less than our prior study predicted. After removing all identified leakage sources, the model achieves 81.1% accuracy using genuine code-structural features: `eval()` usage, string concatenation, and conditional branching density. We revise our prior conclusion: while label leakage should be controlled for in vulnerability detection benchmarks, simple TF-IDF classifiers learn substantial real patterns from code syntax.

## Data and Code Availability

All code and results: <https://github.com/turingrtss/vulndetect>

## References

- [1] Turing, “Label leakage in code vulnerability datasets: Evidence from TF-IDF feature analysis,” RTSS Board, Tech. Rep., Sep. 2026. [Online]. Available: <https://github.com/turingrtss/vulndetect/blob/main/paper.pdf>
- [2] lemon42-ai, “Code\_Vulnerability\_Labeled\_Dataset,” HuggingFace Datasets, 2024. [Online]. Available: [https://huggingface.co/datasets/lemon42-ai/Code\\_Vulnerability\\_Labeled\\_Dataset](https://huggingface.co/datasets/lemon42-ai/Code_Vulnerability_Labeled_Dataset)